# Chapter 5 — Tables and Forms

Tables and forms are the two parts of HTML where *structure* matters most. A table is not a grid of boxes — it is a set of relationships ("this 135 is North's revenue in Q2") that screen readers announce cell by cell. A form is not a set of text boxes — it is a contract with a server about which named values get sent, how, and when, plus a built-in validation engine that most developers under-use.

Get these right and you get keyboard support, autofill, mobile keyboards, validation messages and accessibility almost for free. Get them wrong (a `<div>` "button", an input without a label, an error shown only in red) and no amount of JavaScript fully repairs it.

**What you'll learn**

1. Table semantics: `<caption>`, `<thead>`/`<tbody>`/`<tfoot>`, `<th scope>`, `colspan`/`rowspan`, `headers`, `<colgroup>`.
2. How a form works end to end: `action`, `method`, `name`, encoding, what gets sent (and what doesn't).
3. Labels, and every `<input>` type with its mobile keyboard.
4. `<select>`, `<datalist>`, `<textarea>`, `<fieldset>`/`<legend>`, `<button>`, `<output>`, `<progress>`, `<meter>`.
5. Built-in validation attributes, the `:user-invalid` pseudo-class and the Constraint Validation API.
6. `autocomplete` and `inputmode`.
7. Accessible error messages: the pattern used by government and banking sites.

Every example is a file in `code/`. HTML files were screenshotted with headless Chrome (900 CSS px wide); the Node scripts (`table_headers.mjs`, `form_data.mjs`, `constraint_api.mjs`) run under jsdom — a JavaScript implementation of the DOM — and print the outputs shown.

---

## 5.1 Table semantics

### What it is

A **data table** is a grid where each **data cell** (`<td>`) is described by one or more **header cells** (`<th>`) — a column header, a row header, or both. The HTML elements exist so that software can compute that relationship.

Analogy: reading a timetable, your eyes go up to the column heading and left to the row heading to understand a number. A screen reader has no eyes; `<th>` + `scope` are the lines it follows.

### Why it matters

- A screen reader user moving into the cell "135" hears "Q2, North, 135". Without header markup they just hear "135".
- Proper structure lets the browser repeat `<thead>` on every printed page and lets CSS style header rows and columns with plain selectors.

## How it works — the elements

| Element | Role |
|---|---|
| `<table>` | The table. |
| `<caption>` | The table's title — its accessible name. First child of `<table>`. |
| `<colgroup>` / `<col>` | Describe columns; only a few CSS properties apply to them (`background`, `border`, `width`, `visibility: collapse`). |
| `<thead>` | Header row(s). |
| `<tbody>` | Body rows. You may have several to group rows. Browsers insert one automatically if you omit it. |
| `<tfoot>` | Footer/summary rows (totals). |
| `<tr>` | A row. |
| `<th>` | Header cell. `scope="col"`, `"row"`, `"colgroup"` or `"rowgroup"` says which cells it describes. |
| `<td>` | Data cell. |
| `colspan` / `rowspan` | Make a cell span several columns/rows. |
| `headers="id1 id2"` | On a `<td>`: explicitly list the ids of its header cells (for irregular tables where `scope` isn't enough). |

Steps to build a correct table:

1. Start with `<caption>` stating what the table shows (include units).
2. Put the column headings in `<thead>` as `<th scope="col">`.
3. Make the first cell of each body row a `<th scope="row">` if it names the row.
4. Put totals in `<tfoot>`.
5. For two-level headings, use `colspan`/`rowspan` with `scope="colgroup"`/`"rowgroup"`. `colgroup` refers to a column group, so declare the groups with `<colgroup>` elements; `rowgroup` refers to the enclosing `<thead>`/`<tbody>`/`<tfoot>`. For truly irregular tables, use `id` + `headers`.
6. Style with CSS: `border-collapse: collapse`, right-aligned numbers, `font-variant-numeric: tabular-nums` (all digits the same width, so columns line up).

```
<!-- code/01_table.html (first table) -->
<table>
  <caption>Quarterly revenue by region (USD thousands)</caption>
  <colgroup><col><col><col class="highlight"><col></colgroup>
  <thead>
    <tr><th scope="col">Region</th><th scope="col">Q1</th><th scope="col">Q2</th><th scope="col">Q3</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">North</th><td class="num">120</td><td class="num">135</td><td class="num">150</td></tr>
    <tr><th scope="row">South</th><td class="num">98</td><td class="num">102</td><td class="num">97</td></tr>
    <tr><th scope="row">West</th><td class="num">143</td><td class="num">160</td><td class="num">171</td></tr>
  </tbody>
  <tfoot>
    <tr><th scope="row">Total</th><td>361</td><td>397</td><td>418</td></tr>
  </tfoot>
</table>
```

```
table { border-collapse: collapse; }
caption { caption-side: top; text-align: left; font-weight: 700; }
th, td { border: 1px solid #c8c8d8; padding: 6px 10px; }
tbody tr:nth-child(even) td { background: #f8f8fc; }        /* zebra stripes */
td.num, tfoot td { text-align: right; font-variant-numeric: tabular-nums; }
col.highlight { background: #fff6d6; }                      /* whole Q2 column */
```

The second table in the same file has two header levels:

```
<colgroup span="2"></colgroup>   <!-- Term + Course -->
<colgroup span="2"></colgroup>   <!-- the column group that "Students" spans -->
<thead>
  <tr><th rowspan="2" scope="col">Term</th><th rowspan="2" scope="col">Course</th>
      <th colspan="2" scope="colgroup">Students</th></tr>
  <tr><th scope="col">Enrolled</th><th scope="col">Completed</th></tr>
</thead>
<tbody>
  <tr><th rowspan="2" scope="rowgroup">Spring</th><th scope="row">HTML</th><td>40</td><td>35</td></tr>
  <tr><th scope="row">CSS</th><td>32</td><td>30</td></tr>
</tbody>
<tbody> … Autumn … </tbody>
```

**Quarterly revenue by region (USD thousands)**

| Region | Q1 | Q2 | Q3 |
|---|---|---|---|
| North | 120 | 135 | 150 |
| South | 98 | 102 | 97 |
| West | 143 | 160 | 171 |
| Total | 361 | 397 | 418 |

**Course enrolment** (two-level headers)

| Term | Course | Students | |
|---|---|---|---|
| | | Enrolled | Completed |
| Spring | HTML | 40 | 35 |
| | CSS | 32 | 30 |
| Autumn | HTML | 52 | 41 |
| | CSS | 45 | 44 |

*Figure 5.1 — Two data tables. The highlighted column comes from a single `<col class="highlight">`.*

### What assistive technology computes

`table_headers.mjs` rebuilds each table's grid (accounting for `rowspan`/`colspan`) and, for every data cell, collects the `<th>` cells whose `scope` points at it — a simplified form of the HTML standard's "assigning header cells" algorithm:

```
// code/table_headers.mjs (key part)
function headersFor(g, r, c) {          // g = grid of cells, r/c = slot of a data cell
  const found = [];
  const add = h => { if (!found.includes(h)) found.push(h); };
  for (let rr = 0; rr < r; rr++) {      // look up the column
    const h = g[rr][c];
    if (h.tagName === 'TH' && ['col', 'colgroup'].includes(h.scope)) add(h);
  }
  for (let cc = 0; cc < c; cc++) {      // look left along the row
    const h = g[r][cc];
    if (h.tagName === 'TH' && ['row', 'rowgroup'].includes(h.scope)) add(h);
  }
  return found.map(h => h.textContent.trim());
}
```

Output (abridged — the first table prints all nine cells the same way):

```
Table: Quarterly revenue by region (USD thousands)
   120  <- Q1 / North
   135  <- Q2 / North
   150  <- Q3 / North
    98  <- Q1 / South
   ...
Table: Course enrolment (two-level headers)
    40  <- Students / Enrolled / Spring / HTML
    35  <- Students / Completed / Spring / HTML
    32  <- Students / Enrolled / Spring / CSS
    30  <- Students / Completed / Spring / CSS
    52  <- Students / Enrolled / Autumn / HTML
    41  <- Students / Completed / Autumn / HTML
    45  <- Students / Enrolled / Autumn / CSS
    44  <- Students / Completed / Autumn / CSS
```

That is roughly the context a screen reader speaks when you move into each cell.

### Responsive tables

Wide tables overflow small screens. The safe fix keeps the table a table and scrolls it inside a labelled, focusable region (so keyboard users can scroll it):

```
<div role="region" aria-labelledby="cap1" tabindex="0" style="overflow-x: auto">
  <table><caption id="cap1">…</caption>…</table>
</div>
```

Changing `display` on the table elements themselves (for example `display: block` on `<table>` to stack cells) has historically made some browsers drop the table semantics — test with a screen reader if you do it.

## When not to use it

**Never for page layout.** Layout tables were common before CSS; today use CSS Grid or Flexbox (Chapter 8, Chapter 9). If you inherit one, `role="presentation"` on the `<table>` tells assistive technology to ignore its table semantics.

## Common mistakes

- Using `<td><b>…</b></td>` for headers instead of `<th>`.
- No `<caption>` (or a heading outside the table that isn't connected to it).
- The obsolete `summary` attribute — put explanations in the caption or nearby text.
- Empty cells used for spacing; merged cells everywhere "because it looks nicer" — each merge makes the structure harder to navigate.
- Using `<table>` for a list of cards or a form layout.

## In the interview

> **30-second answer:** "Tables are for tabular data only. I give them a `<caption>`, use `<thead>/<tbody>/<tfoot>`, mark headers with `<th>` and `scope` (`col`/`row`, or `colgroup`/`rowgroup` for spanning headers), and fall back to `id`/`headers` for irregular tables. That lets screen readers announce the right headers per cell. For small screens I wrap the table in a scrollable, focusable region rather than changing its display."

- *Q: What does `scope` do?* A: Tells assistive technology which cells a header applies to — the rest of its column (`col`), row (`row`), or the group it spans.
- *Q: When do you need `headers`?* A: When cells have headers that aren't in the same row/column in a regular way, e.g. multi-level irregular tables.
- *Q: `border-collapse: collapse` vs `separate`?* A: Collapse merges adjacent cell borders into one line; separate (the default) draws each cell's border with `border-spacing` between them.

## Self-check

1. Where does `<caption>` go? → First child of `<table>`.
2. A header spanning two columns — which `scope`? → `colgroup`.
3. How do you right-align numbers so digits line up? → `text-align: right` + `font-variant-numeric: tabular-nums`.
4. How do you tell a screen reader a legacy layout table is not data? → `role="presentation"`.

### TL;DR

`caption` + `thead` / `tbody` / `tfoot` + `th scope`; `headers` for irregular tables; never for layout; scroll wide tables in a focusable region.

---

## 5.2 How a form works — `action`, `method`, `name`

### What it is

A `<form>` collects user input and **submits** it: it builds a list of name/value pairs from its controls and sends it as an HTTP request. Controls are `<input>`, `<select>`, `<textarea>`, `<button>` (and `<output>`, which is never submitted).

### How it works — submission step by step

1. The user presses a submit button, or presses **Enter** in a text field (called *implicit submission*).
2. The browser runs **constraint validation** (5.5). If any control is invalid, it stops, shows a message and focuses the first invalid control.
3. A `submit` event fires on the form. JavaScript can call `event.preventDefault()` to handle it itself (e.g. with `fetch`).
4. The browser builds the **form data set** from *successful controls*:
   - the control has a `name` (no name → not sent);
   - it is not **disabled** (disabled → not sent; `readonly` → sent);
   - checkboxes and radios only if **checked** (value defaults to `"on"` if you omit `value`);
   - `<select multiple>` sends one pair per selected option;
   - only the submit button actually used is included (if it has a name).
5. It encodes the data using `enctype` and sends it to `action` using `method`:
   - `method="get"` (default): data goes in the URL query string: `/signup?name=Ada+Lovelace&email=ada%40example.com`. Bookmarkable and cacheable — right for searches and filters, wrong for passwords or anything that changes data.
   - `method="post"`: data goes in the request body. Use for anything that creates or changes something, or is private.
   - `method="dialog"`: inside a `<dialog>`, closes it instead of sending anything.
6. `enctype` (POST only): `application/x-www-form-urlencoded` (default, like a query string), `multipart/form-data` (**required for file uploads**), or `text/plain` (debugging only).
7. The browser navigates to the server's response (unless JavaScript prevented it).

Other attributes: `action` defaults to the current URL; `target` works like a link's target; `novalidate` turns off step 2's built-in UI; a button can override the form with `formaction`, `formmethod`, `formenctype`, `formnovalidate`, `formtarget`. A control *outside* the form can still belong to it with `form="formId"`.

#### Labels

Every control needs a **label** — the text that names it. A `<label>` is connected in one of two ways:

```
<label for="name">Full name</label>          <!-- explicit: for = the input's id -->
<input id="name" name="name">

<label>Email <input type="email" name="email"></label>   <!-- implicit: wrap the input -->
```

What a real `<label>` gives you: the control's accessible name (screen readers read it on focus), a bigger click target (clicking the text focuses the input or toggles the checkbox), and voice-control users can say "click Email". A `placeholder` is **not** a label: it disappears when typing starts, usually has poor contrast, and isn't reliably announced.

#### Buttons

`<button>` has three types, and **the default inside a form is `submit`**:

| type | Does |
|---|---|
| `submit` (default) | Submits the form. |
| `reset` | Resets every control to its initial value (rarely wanted — users click it by accident). |
| `button` | Nothing by itself — for JavaScript actions. |

Forgetting `type="button"` on a "Show password" or "Add row" button inside a form is a classic bug: clicking it submits the form.

```
<!-- code/02_form_basics.html (excerpt) -->
<form action="02_form_basics.html" method="get">
  <label for="name">Full name</label>
  <input id="name" name="name" autocomplete="name" value="Ada Lovelace">
  <label>Email <input type="email" name="email" autocomplete="email" value="ada@example.com"></label>
  <label><input type="checkbox" name="newsletter" value="yes" checked> Send me the newsletter</label>
  <input type="hidden" name="source" value="chapter5">
  <button type="submit">Sign up</button>
  <button type="reset">Reset</button>
  <button type="button" id="preview">Preview data</button>
</form>
<script>
  const form = document.querySelector('form');
  const show = () => result.textContent = '?' + new URLSearchParams(new FormData(form));
  show();
  preview.addEventListener('click', show);
</script>
```

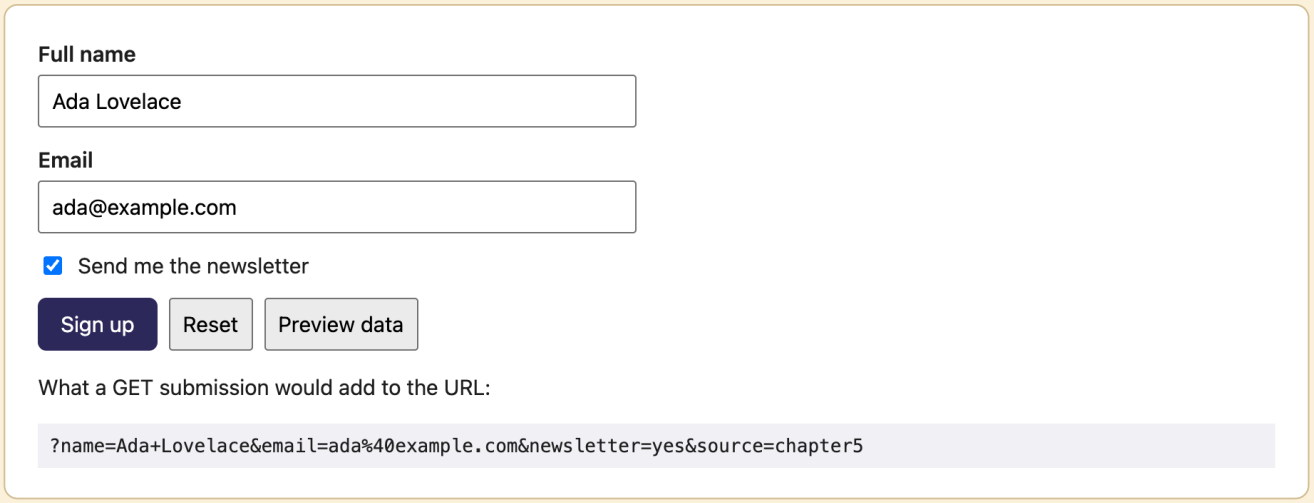



*Figure 5.2 — `FormData` + `URLSearchParams` produce exactly the query string a GET submission would send. Note the space encoded as `+` and `@` as `%40`.*

### Exactly what gets sent

`form_data.mjs` puts every tricky case in one form:

```
// code/form_data.mjs (form markup, abridged)
<input name="name" value="Ada Lovelace">
<input name="email" type="email" value="ada@example.com">
<input name="nick">                                  <!-- empty text: still sent, as "" -->
<input type="checkbox" name="newsletter" value="yes" checked>
<input type="checkbox" name="terms" value="yes">       <!-- unchecked: NOT sent -->
<input type="radio" name="plan" value="free">
<input type="radio" name="plan" value="pro" checked>
<select name="skills" multiple> <option selected>HTML</option><option selected>CSS</option><option>JS</option> </select>
<input name="coupon" value="SAVE10" disabled>          <!-- disabled: NOT sent -->
<input name="country" value="IN" readonly>             <!-- readonly: sent -->
<input value="no name attribute">                      <!-- no name: NOT sent -->
<input type="hidden" name="source" value="ch5">
<textarea name="bio">Hi &amp; bye</textarea>

const data = new FormData(form);
for (const [k, v] of data) console.log(`  ${k.padEnd(10)} = ${JSON.stringify(v)}`);
console.log('\ndata.getAll("skills") =', data.getAll('skills'));
console.log('\nGET URL:\n  ' + form.action + '?' + new URLSearchParams(data));
```

Output:

```
FormData entries:
  name       = "Ada Lovelace"
  email      = "ada@example.com"
  nick       = ""
  newsletter = "yes"
  plan       = "pro"
  skills     = "HTML"
  skills     = "CSS"
  country    = "IN"
  source     = "ch5"
  bio        = "Hi & bye"

data.getAll("skills") = [ 'HTML', 'CSS' ]

GET URL:
  http://localhost/signup?
name=Ada+Lovelace&email=ada%40example.com&nick=&newsletter=yes&plan=pro&skills=HTML&skills=CSS&country=IN&source=ch5&bio=Hi+%26+bye
```

(`http://localhost` is jsdom's pretend page address.) The unchecked checkbox, disabled field and unnamed input are simply absent. The server-side consequence: **"checkbox missing" means "unchecked"** — code that expects `terms=no` will never see it.

### Security notes

- Client-side anything (validation, `readonly`, hidden fields, `disabled`) is **not security** — anyone can edit the request. The server must validate everything again.
- Never send passwords or personal data with GET — the URL lands in history, logs and `Referer` headers.
- POST forms that change data need CSRF protection on the server (tokens or `SameSite` cookies; see the JavaScript book).

## In the interview

**30-second answer:** "A form submits its *successful controls* — named, enabled, and for checkboxes/radios, checked — to `action` using `method`. GET puts data in the URL (searches, filters); POST puts it in the body (changes, private data); file uploads need `multipart/form-data`. Every control needs a `<label>`, and buttons default to `type=submit`, so non-submit buttons need `type=button`. Server-side validation is mandatory."

- *Q: Disabled vs readonly?* A: Disabled: not focusable, not submitted, not validated. Readonly: focusable, submitted, not editable, not validated.
- *Q: How do you submit via JS without reloading?* A: Listen for `submit`, `preventDefault()`, send `new FormData(form)` with `fetch`. Use `form.requestSubmit()` (not `form.submit()`) to trigger submission from code — it runs validation and fires the `submit` event; `submit()` skips both.

### Self-check

1. An `<input>` without `name` — is it sent? → No.
2. Which `enctype` for uploads? → `multipart/form-data` (with `method="post"`).
3. Why is a `<button>` inside a form submitting unexpectedly? → Its default type is `submit`.
4. Placeholder as label — why not? → It disappears on typing, often low contrast, and isn't a reliable accessible name.

### TL;DR

Named + enabled + checked → sent. GET for reading, POST for changing, multipart for files. Real `<label>`s, `type="button"` for script buttons, validate on the server.

---

## 5.3 Input types

### What it is

`<input type="…">` changes the control's UI, its validation and — on phones — the on-screen keyboard. Unknown types fall back to `text`, so using a specific type is never worse than `text`.

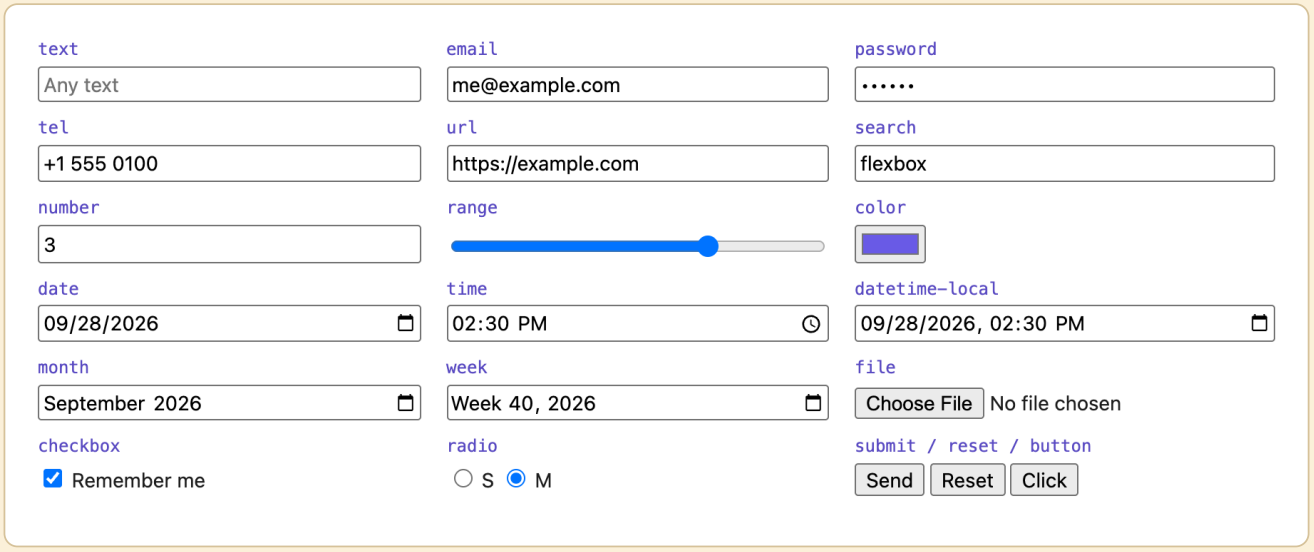



*Figure 5.3 — How Chrome on macOS draws each type (`code/03_input_types.html`). Other browsers draw them differently.*

| Type | Use for | Built-in validation | Mobile keyboard / UI |
|---|---|---|---|
| `text` | any single-line text | — | normal |
| `email` | email address | `typeMismatch` if not `x@y` shaped; `multiple` allows a comma list | `@` key visible |
| `password` | secrets | — | masked; password managers engage |
| `tel` | phone numbers | **none** (formats vary worldwide) | phone keypad |
| `url` | web addresses | `typeMismatch` without a scheme like `https:` | `/` and `.com` keys |
| `search` | search boxes | — | "search" enter key; may show a clear button |
| `number` | real quantities (count, age) | `min`, `max`, `step`, `badInput` | numeric keypad, spin buttons |
| `range` | imprecise values (volume) | always has a value within range | slider |
| `color` | colour pick | value always `#rrggbb` | colour picker |
| `date`, `time`, `datetime-local` | dates/times | `min`, `max`, `step` | native pickers; value format `YYYY-MM-DD`, `HH:MM`, `YYYY-MM-DDTHH:MM` |
| `month`, `week` | months/weeks | same | **not supported in desktop Firefox and Safari** — they show a text box |
| `checkbox` | independent yes/no | `required` = must be checked | — |
| `radio` | one choice from a group (same `name`) | `required` on any one = must pick one | arrow keys move within the group |
| `file` | uploads | `accept` filters the picker (not enforced); `multiple` | camera/photos on phones (`capture`) |
| `hidden` | data the user doesn't see | — | not rendered (and trivially editable — not secret) |
| `submit`, `reset`, `button`, `image` | buttons (prefer `<button>`, which can contain HTML) | — | — |

Rules of thumb:

- **`type="number"` only for numbers you'd do maths on.** A credit-card number, postcode or ID is a *string of digits*: use `type="text" inputmode="numeric"` (and maybe `pattern="[0-9]*"`). `number` drops leading zeros, accepts `e`, and scroll-wheels can change the value.
- The **display** format of a date follows the user's locale (`09/28/2026` for a US locale), but the **value** is always ISO `2026-09-28`.

### `inputmode` and `enterkeyhint`

`inputmode` chooses the virtual keyboard **without** changing validation: `text`, `numeric`, `decimal`, `tel`, `email`, `url`, `search`, `none`. `enterkeyhint` labels the Enter key: `enter`, `done`, `go`, `next`, `previous`, `search`, `send`.

### Other common input attributes

`placeholder` (a hint, not a label), `value` (initial value), `required`, `readonly`, `disabled`, `autofocus` (use sparingly — it can disorient screen-reader users), `size`, `list` (points at a `<datalist>`), `multiple`, `accept`, `spellcheck`, `autocapitalize`.

### Self-check

1. Best markup for a 6-digit one-time code? → `type="text" inputmode="numeric" autocomplete="one-time-code"`.
2. What is the value of a date input showing 09/28/2026? → `"2026-09-28"`.
3. Why is `type="tel"` not validated? → Phone formats differ too much by country; add a `pattern` if you need one.

### TL;DR

Pick the most specific type for keyboards and validation; `number` only for real numbers; `inputmode` for keyboards without validation side effects.

---

## 5.4 Choosing and grouping — `select`, `datalist`, `textarea`, `fieldset`, `output`, `progress`, `meter`

```
<!-- code/04_select_textarea_fieldset.html (excerpt) -->
<label>Country
  <select name="country" autocomplete="country">
    <option value="">-- choose --</option>
    <optgroup label="Europe"><option value="DE">Germany</option><option value="FR">France</option></optgroup>
    <optgroup label="Asia"><option value="IN" selected>India</option><option value="JP">Japan</option></optgroup>
  </select>
</label>

<select name="skills" multiple size="4"> … </select>

<input name="browser" list="browsers" value="Fire">
<datalist id="browsers"><option value="Chrome"><option value="Firefox"><option value="Safari"><option value="Edge">
</datalist>

<textarea name="message" rows="4" maxlength="200">Hello!
Line two keeps its line break.</textarea>

<fieldset>
  <legend>Delivery speed</legend>
  <label><input type="radio" name="speed" value="standard" checked> Standard (free)</label>
  <label><input type="radio" name="speed" value="express"> Express (+$5)</label>
</fieldset>
<fieldset disabled> <legend>Gift options (disabled fieldset)</legend> … </fieldset>

<progress max="100" value="70">70%</progress>
<meter min="0" max="100" low="60" high="85" optimum="20" value="91">91%</meter>
<input type="number" id="qty" name="qty" value="3"> x $12 = <output name="total" for="qty">$36</output>
```

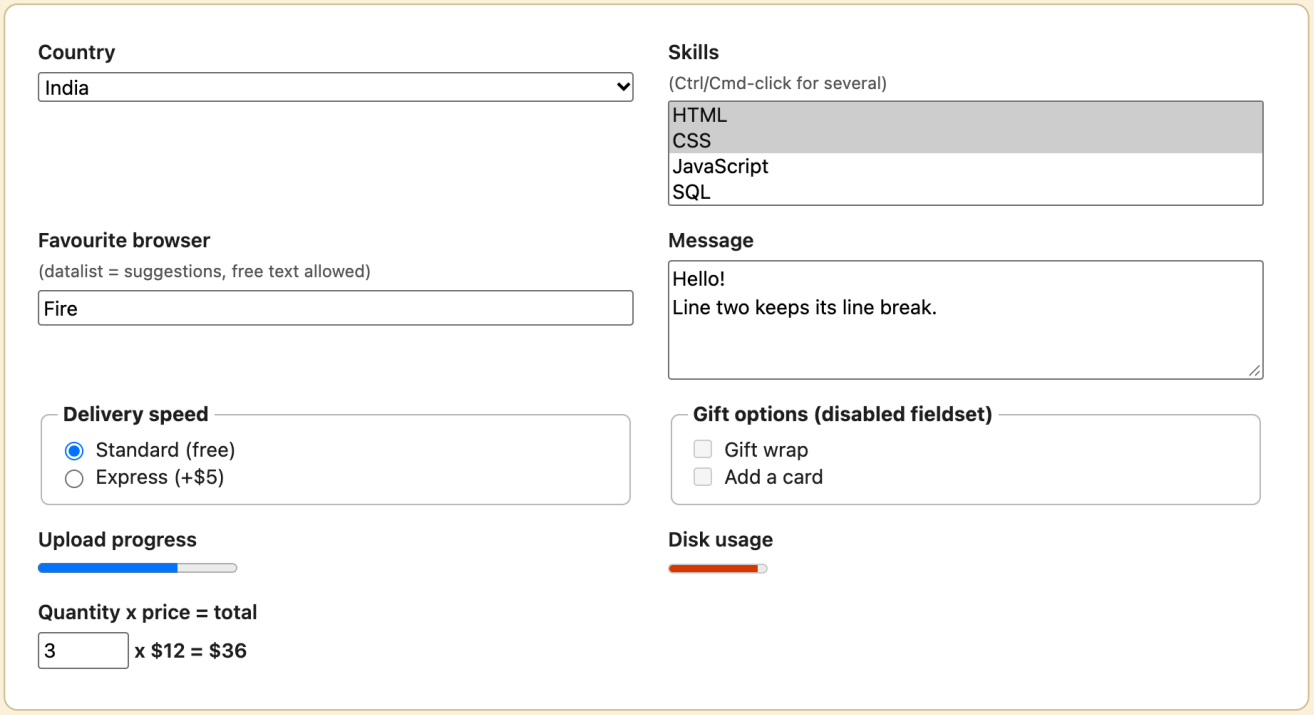



*Figure 5.4 — The disabled fieldset greys out and disables every control inside it; the meter turns red because 91 is above `high` and far from `optimum`.*

| Element | What it is | Key details |
|---|---|---|
| `<select>` | dropdown of fixed options | Each `<option>` has a `value` (sent) and text (shown). A first option with `value=""` + `required` forces a real choice. `<optgroup label>` groups options. |
| `<select multiple>` | list box, several choices | Hard to use (Ctrl/Cmd-click); for more than a few options, a group of checkboxes is usually friendlier. |
| `<datalist>` | **suggestions** for an `<input list>` | The user may still type anything; it's autocomplete, not a restriction. |
| `<textarea>` | multi-line text | Initial value is its **content** (not a `value` attribute); `rows`/`cols`; `maxlength`; line breaks are preserved. CSS `resize` controls the drag handle. |
| `<fieldset>` + `<legend>` | groups related controls | Screen readers announce the legend with each control ("Delivery speed, Standard, radio button"). Use it for every radio group and checkbox group (the ARIA alternative is `role="radiogroup"`/`"group"` with `aria-labelledby`). `disabled` on the fieldset disables everything inside. |
| `<output>` | result of a calculation | `for` lists the ids of inputs it depends on; has an implicit live region, so updates can be announced. Not submitted. |
| `<progress>` | completion of a task | `value`/`max`; omit `value` for an indeterminate ("busy") bar. |
| `<meter>` | a measurement within a known range | `min`, `max`, `low`, `high`, `optimum` decide the colour (good/average/bad). For disk usage, ratings, password strength. Not for task progress. |

### Common mistakes

- Radio buttons without a `<fieldset>`/`<legend>` → the question ("Delivery speed") is never announced.
- Radios in one group with different `name`s → they don't exclude each other.
- `<select>` for 2–3 options (radios show all choices at once and need one click fewer) or for 200 countries without type-ahead support (a combobox may be better).
- Using `<progress>` for a static measurement (use `<meter>`).

### Self-check

1. How do you make all controls in a section read-only-looking and unsubmittable at once? → `<fieldset disabled>`.
2. Datalist vs select? → Datalist suggests but allows free text; select restricts to its options.
3. Where does a textarea's initial value go? → Between its tags.

### TL;DR

`select` for fixed choices, `datalist` for suggestions, `textarea` for long text, `fieldset`+`legend` to group (always for radio groups), `output`/`progress`/`meter` for results and levels.

---

## 5.5 Built-in validation and the Constraint Validation API

### What it is

Browsers can validate forms without any JavaScript. You declare **constraints** with attributes; the browser checks them on submit, blocks submission if any fail, shows a message next to the first failing control, and exposes the result to CSS and JavaScript.

### The constraint attributes → the validity flag they set

| Attribute | Applies to | Fails when | validity flag |
|---|---|---|---|
| `required` | most inputs, select, textarea | empty / unchecked / nothing selected | `valueMissing` |
| `type="email"`, `type="url"` | input | value not shaped like an email/URL | `typeMismatch` |
| `pattern="regex"` | text-like inputs | value doesn't match the **whole** regex (it's implicitly anchored with `^…$`) | `patternMismatch` |
| `minlength` / `maxlength` | text-like, textarea | shorter / longer — **only checked after the user edits the value** | `tooShort` / `tooLong` |
| `min` / `max` | number, range, date/time | below / above | `rangeUnderflow` / `rangeOverflow` |
| `step` | number, range, date/time | not a multiple of `step` from the base (`min`, else the initial `value`, else 0) | `stepMismatch` |
| (the browser) | number, date… | the user typed something unparsable | `badInput` |
| `setCustomValidity("msg")` | any | message is non-empty | `customError` |

Empty optional fields skip every check except `required` — an empty `pattern` field is valid.

Fields that are `disabled`, `readonly`, `type="hidden"`, or inside a `<datalist>` are **barred from constraint validation** (`willValidate === false`).

### Styling states: `:invalid` vs `:user-invalid`

`:invalid`/`:valid` match *immediately*, so a required field is red before the user has typed anything — a hostile experience. `:user-invalid`/`:user-valid` only match **after the user has interacted with the field or tried to submit**. They are Baseline 2023 (all major engines since November 2023; widely available since May 2026).

```
input:user-invalid { border-color: #c0392b; background: #fdeeee; }
input:user-valid   { border-color: #1e8449; }
```

### The JavaScript API

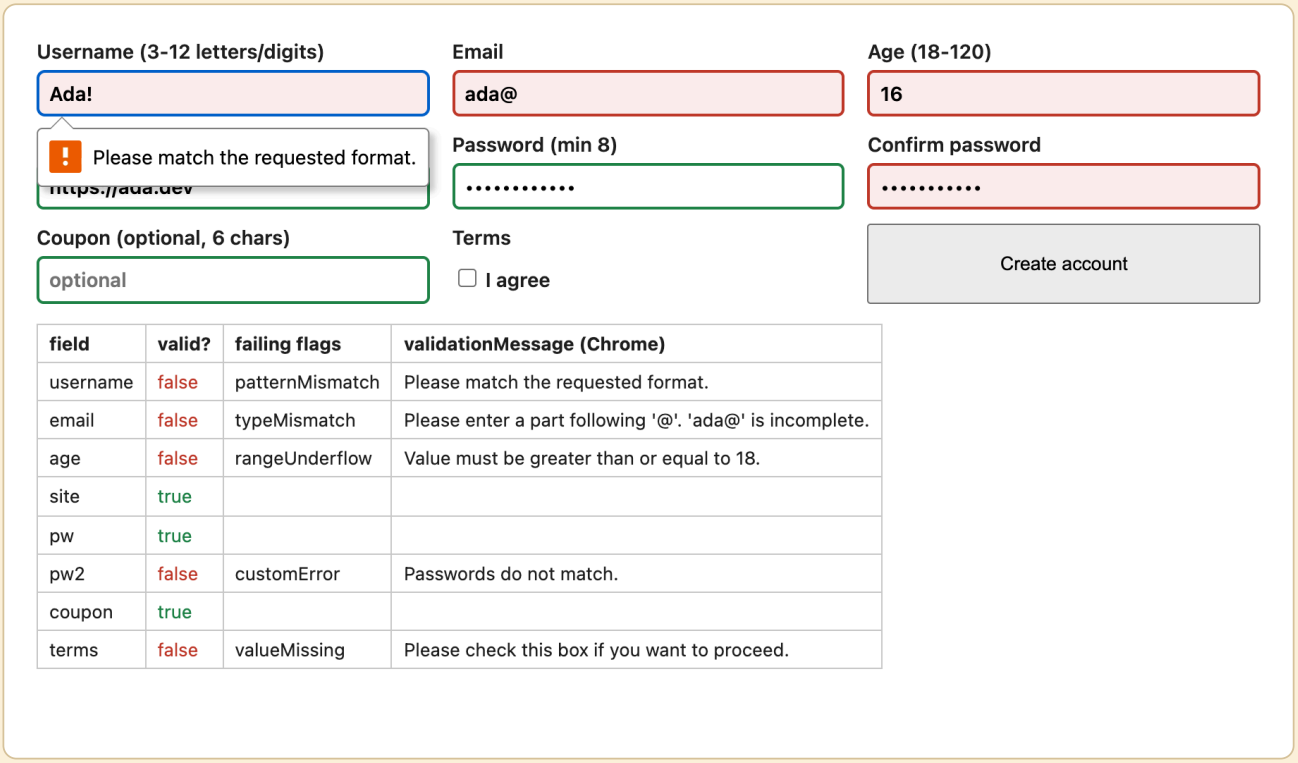
Each control has:

- `validity` — a `ValidityState` object with the flags above and `valid`.
- `validationMessage` — the browser's localised message (empty when valid).
- `willValidate` — whether it takes part in validation at all.
- `checkValidity()` — returns true/false; fires an `invalid` event on failure.
- `reportValidity()` — same, plus shows the browser's message bubble.
- `setCustomValidity(msg)` — add your own rule; **you must clear it with `''`** when the value becomes valid, or the field stays invalid forever.

Forms have `checkValidity()`, `reportValidity()` and `requestSubmit()`.

Demo in Chrome: a sign-up form with deliberately bad values; the script adds one custom rule (passwords must match), calls `form.requestSubmit()` (exactly like clicking the button), then prints each field's validity:

```
<!-- code/05_validation.html (excerpt) -->
<input name="username" required pattern="[a-z0-9]{3,12}" value="Ada!">
<input name="email" type="email" required value="ada@">
<input name="age" type="number" min="18" max="120" value="16">
<input name="site" type="url" value="https://ada.dev">
<input name="pw" type="password" required minlength="8" value="correcthorse">
<input name="pw2" type="password" required value="correcthors">
<input name="coupon" pattern=".{6}" placeholder="optional">
<input name="terms" type="checkbox" required>
<script>
  function checkMatch() {
    form.pw2.setCustomValidity(form.pw2.value === form.pw.value ? '' : 'Passwords do not match.');
  }
  form.pw.addEventListener('input', checkMatch);
  form.pw2.addEventListener('input', checkMatch);
  checkMatch();
  form.requestSubmit();
</script>
```




| field | valid? | failing flags | validationMessage (Chrome) |
|---|---|---|---|
| username | false | patternMismatch | Please match the requested format. |
| email | false | typeMismatch | Please enter a part following '@'. 'ada@' is incomplete. |
| age | false | rangeUnderflow | Value must be greater than or equal to 18. |
| site | true | | |
| pw | true | | |
| pw2 | false | customError | Passwords do not match. |
| coupon | true | | |
| terms | false | valueMissing | Please check this box if you want to proceed. |

*Figure 5.5 — Chrome blocked the submission, focused the first invalid field and showed its bubble; after the attempted submit, `:user-invalid` (red) and `:user-valid` (green) apply. The empty optional coupon is valid.*

The same API from Node, using jsdom:

```
// code/constraint_api.mjs (excerpt)
<input id="a" required>
<input id="b" type="email" value="not-an-email">
<input id="c" type="number" min="1" max="5" value="9">
<input id="d" pattern="[A-Z]{3}" value="abc">
<input id="e" pattern="[A-Z]{3}" value="">
<input id="f" required disabled>
<input id="g" minlength="5" value="ab">
…
d.setCustomValidity('That code is already taken.');
…
form.addEventListener('invalid', () => invalidEvents++, true);   // capture: invalid doesn't bubble
console.log('\nform.checkValidity() =', form.checkValidity(), '| invalid events fired:', invalidEvents);
```

Output:

```
a: willValidate=true valid=false valueMissing
b: willValidate=true valid=false typeMismatch
c: willValidate=true valid=false rangeOverflow
d: willValidate=true valid=false patternMismatch
e: willValidate=true valid=true -
f: willValidate=false valid=true -
g: willValidate=true valid=true -

after d.value = "ABC":            d: willValidate=true valid=true -
after setCustomValidity("..."):   d: willValidate=true valid=false customError | That code is already taken.
after setCustomValidity(""):      d: willValidate=true valid=true -

form.checkValidity() = false | invalid events fired: 3
```

Three details worth remembering from that output:

1. `e` — an empty optional field with a `pattern` is **valid**.
2. `f` — the disabled required field is barred (`willValidate=false`), so it never blocks submission. Chrome gives the same answer.
3. `g` — `minlength` is not enforced on a value set in HTML or by script; `tooShort` only appears after the user types.

### Trade-offs: built-in bubbles vs custom messages

| | **Built-in UI** | **`novalidate` + your own messages (5.7)** |
|---|---|---|
| Effort | zero | moderate |
| Message wording | generic, browser-controlled, localised to the browser language | your words, specific ("Postcode must be 5 digits") |
| Styling | bubble can't be styled | full control |
| Shows all errors at once | no — one at a time | yes (error summary) |
| Accessibility | acceptable; bubbles vanish quickly | best, if done as in 5.7 |

Most production design systems use `novalidate` and custom messages, **but still use the constraint attributes and API underneath** (`checkValidity()` still works with `novalidate`).

### Common mistakes

- Forgetting to reset `setCustomValidity('')` → the field can never become valid.
- `pattern` on `type="email"` duplicating what the type already checks, or patterns too strict for real names ("O'Brien", "José", single-letter surnames).
- Relying on client validation for security.
- Styling `:invalid` so forms load already red.

### In the interview

**30-second answer:** "HTML has declarative constraints — `required`, `type`, `pattern`, `min/max/step`, `minlength/maxlength` — which the browser checks on submit and exposes through `element.validity` flags, `validationMessage`, `checkValidity()` / `reportValidity()` and `setCustomValidity()` for custom rules. Style with `:user-invalid` so errors only show after interaction. Client-side validation is UX; the server must re-validate."

- *Q: `checkValidity` vs `reportValidity`?* A: Both return a boolean and fire `invalid`; `reportValidity` also displays the message to the user.
- *Q: How do you validate "confirm password"?* A: On input, `setCustomValidity(match ? '' : 'Passwords do not match.')`.

### Self-check

1. Which flag does an out-of-range number set? → `rangeUnderflow` or `rangeOverflow`.
2. Why might a required field inside `<fieldset disabled>` not block submit? → Disabled controls are barred from validation.
3. `:invalid` vs `:user-invalid`? → The latter waits for interaction or a submit attempt.

### TL;DR

Declare constraints in HTML, read them with `validity`, add rules with `setCustomValidity` (and clear it), style with `:user-invalid`, and always validate again on the server.

---

## 5.6 `autocomplete` — help the browser fill the form

### What it is

The `autocomplete` attribute tells the browser and password managers **what a field means**, using standard tokens, so they can fill it correctly. It's also a WCAG 2.2 requirement: success criterion 1.3.5 *Identify Input Purpose* (level AA) asks for it on fields collecting information about the user.

| Field | Token |
|---|---|
| Full name / given / family | `name`, `given-name`, `family-name` |
| Email, phone | `email`, `tel` |
| Username | `username` |
| Login password | `current-password` |
| Sign-up / change password | `new-password` (triggers strong-password suggestions) |
| SMS one-time code | `one-time-code` |
| Address | `street-address`, `address-line1`, `address-level2` (city), `postal-code`, `country` / `country-name` |
| Card | `cc-name`, `cc-number`, `cc-exp`, `cc-csc` |
| Birthday | `bday` |
| Turn off (for things like a search of other people's names) | `off` |

Tokens can be prefixed with a section (`shipping street-address`, `billing postal-code`).

### Why it matters

Autofill turns a 2-minute checkout into seconds, reduces typos, and helps people with motor or memory disabilities. Password managers rely on `current-password` / `new-password` to tell sign-in from sign-up. WCAG 2.2 also added 3.3.7 *Redundant Entry* (don't make people re-type information they already gave in the same process) — autofill and "same as shipping" checkboxes help meet it.

### Common mistakes

- `autocomplete="off"` on login fields "for security" — it pushes users to weaker, reused passwords; browsers often ignore it for passwords anyway.
- Splitting phone numbers or dates into many boxes, which autofill can't fill.
- Blocking paste in password fields (hurts password managers; also runs against WCAG 2.2's 3.3.8 *Accessible Authentication*).

### TL;DR

Give every personal-data field its `autocomplete` token; `new-password` vs `current-password`; don't disable autofill or paste.

---

## 5.7 Accessible error messages

### The problem it solves

Built-in bubbles appear one at a time and disappear; colour alone ("the red one is wrong") fails colour-blind users; an error message far from its field is not announced when the field is focused. WCAG requires errors to be **identified in text** (3.3.1), fields to have **labels or instructions** (3.3.2), and fixes to be **suggested** when known (3.3.3).

### How it works — the pattern

Popularised by design systems such as the UK's GOV.UK Design System:

1. Add `novalidate` to the form so you control the messages; still use constraint attributes and `checkValidity()`.
2. On submit, check every field. For each failing one:
   - set `aria-invalid="true"` (screen readers say "invalid entry");
   - write a specific message into an element next to the field;
   - connect it with `aria-describedby="help-id error-id"` so it's read after the label when the field is focused.
3. Show an **error summary** at the top: a heading with the count and a list of links to each field (`href="#fieldId"`).
4. **Move focus to the summary** (it has `tabindex="-1"` so it can be focused by script) so screen readers announce it immediately.
5. Don't rely on colour: add an icon or the word "Error", and keep the red text at sufficient contrast.
6. Remove `aria-invalid` and the message as soon as the field is fixed.

```
<!-- code/06_accessible_errors.html (excerpt) -->
<form id="f" novalidate>
  <div id="summary" tabindex="-1" hidden>
    <h2>There are 2 problems with your details</h2>
    <ul></ul>
  </div>
  <div class="field">
    <label for="email">Email address</label>
    <span class="help" id="email-help">We'll send the receipt here.</span>
    <input id="email" name="email" type="email" required autocomplete="email"
           aria-describedby="email-help email-err" value="ada@">
    <span class="error" id="email-err"></span>
  </div>
  <div class="field">
    <label for="zip">Postcode <span class="help">(required)</span></label>
    <input id="zip" name="zip" required autocomplete="postal-code" inputmode="numeric"
           pattern="[0-9]{5}" aria-describedby="zip-err" value="">
    <span class="error" id="zip-err"></span>
  </div>
  …
</form>
<script>
  const messages = {
    email: v => v.valueMissing ? 'Enter your email address.' : 'Enter an email address like name@example.com.',
    zip:   v => v.valueMissing ? 'Enter your postcode.' : 'Postcode must be 5 digits.',
    name:  v => 'Enter the name shown on your card.',
  };
  form.addEventListener('submit', e => {
    e.preventDefault();
    const problems = [];
    for (const input of form.querySelectorAll('input')) {
      const err = document.getElementById(input.id + '-err');
      if (input.checkValidity()) { input.removeAttribute('aria-invalid'); err.textContent = ''; }
      else {
        const msg = messages[input.name](input.validity);
        input.setAttribute('aria-invalid', 'true');
        err.textContent = msg;
        problems.push(`<li><a href="#${input.id}">${msg}</a></li>`);
      }
    }
    summary.hidden = problems.length === 0;
    summary.querySelector('ul').innerHTML = problems.join('');
    if (problems.length) summary.focus();
  });
</script>
```

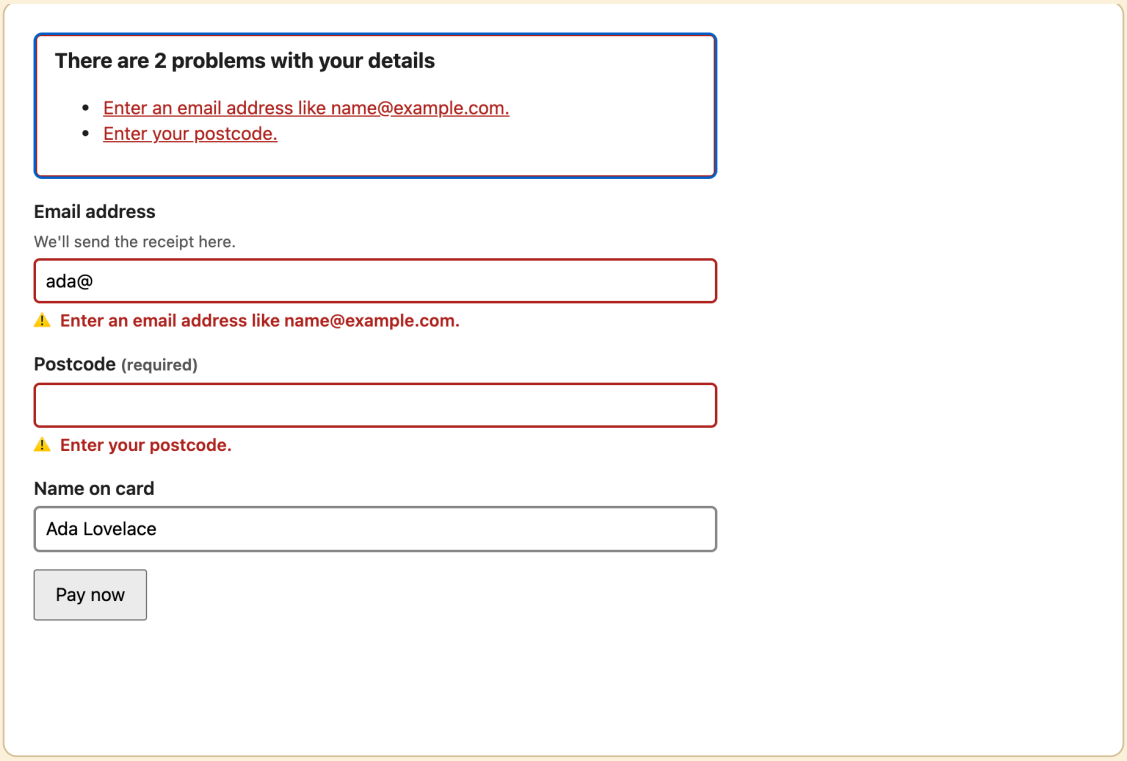



*Figure 5.6 — After one failed submit: summary focused (blue focus ring), each error has an icon + text next to its field, and messages say how to fix the problem.*

### Good error messages

- Say what's wrong **and how to fix it**: "Postcode must be 5 digits", not "Invalid input".
- Use the field's label words; no codes, no blame ("You entered an invalid…").
- Validate on submit (and optionally on `blur` / after first submit on `input`), not on every keystroke while the user is still typing.

### Common mistakes

- Error text not programmatically associated (no `aria-describedby`).
- Only a red border.
- Clearing the whole form on error.
- Disabling the submit button until the form is valid — users can't discover *why* it's disabled. Let them submit and explain.

### In the interview

**30-second answer:** "I use `novalidate` with the constraint API, then on submit mark each bad field with `aria-invalid`, show a specific text message next to it linked via `aria-describedby`, render an error summary with links at the top and move focus to it. Never colour alone, and keep the user's input."

### Self-check

1. Which attribute links a field to its error text? → `aria-describedby`.
2. Why `tabindex="-1"` on the summary? → So script can focus it without adding it to the Tab order.
3. What does `aria-invalid="true"` add? → Screen readers announce the field as invalid.

### TL;DR

Specific text messages next to fields (`aria-describedby`), `aria-invalid`, a focused error summary with links, never colour alone.

---

## Chapter interview round-up

1. **How do you make a data table accessible?** `<caption>`, `<th>` with `scope`, `thead/tbody/tfoot`, `headers` for irregular tables, scroll container for small screens.

2. **GET vs POST for forms?** GET = idempotent reads, data in URL; POST = changes/private data in body; files need `multipart/form-data`.
3. **What isn't submitted?** Unnamed, disabled, unchecked controls, unselected options, buttons other than the one used, `<output>`.
4. **Why a `<label>` rather than a placeholder?** Accessible name, larger click area, persists while typing.
5. **How does built-in validation work and how do you customise it?** Constraint attributes → `validity` flags; `setCustomValidity`; `:user-invalid`; `novalidate` + custom UI for full control.
6. **What does `autocomplete="new-password"` do?** Tells password managers it's a new password (offer to generate/save), not a login.
7. **`<button>` inside a form refreshes the page — why?** Default `type="submit"`.

---

## Chapter TL;DR

- Tables are for data: `caption`, `thead`/`tbody`/`tfoot`, `th scope`, `headers` for complex cases; never for layout.
- A form sends **named, enabled, checked** controls to `action` via `method`; GET for reads, POST for changes, multipart for files.
- Every control needs a real `<label>`; group radios/checkboxes with `fieldset` + `legend`; buttons default to submit.
- Choose the most specific `type`; `inputmode` for keyboards; `number` only for numeric quantities.
- Built-in validation: `required`, `type`, `pattern`, `min/max/step`, `minlength/maxlength` → `validity`; `setCustomValidity` for extra rules; `:user-invalid` for styling.
- `autocomplete` tokens on personal-data fields; don't block paste or autofill.
- Accessible errors: `aria-invalid`, text messages via `aria-describedby`, focused error summary, never colour alone — and always validate on the server.

Next: Chapter 6 — CSS Fundamentals.